MDPI

Article

Evaluating the Impact of Adapter-Based Fine-Tuning on Structured Parsing Performance in Large Language Models

Ratomir Karlović [1], Luka Sever [1], Sandi Berossi Šegota [1] and Ivan Lorencin [1]

[1] Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia; ratomir.karlovic@unipu.hr (R.K.), luka.sever@unipu.hr (L.S.), sandi.baressi.segota@unipu.hr (S.B.S), ivan.lorencin@unipu.hr (I.L.)

* Correspondence: ivan.lorencin@unipu.hr;

Abstract

Recent advances in large language models (LLMs) highlight two dominant strategies for performance improvement: prompt engineering and fine-tuning. While prompt design can significantly influence model output, it remains uncertain whether lightweight fine-tuning methods,such as adapter-based training, offer meaningful advantages for structured, domain-specific tasks. This study builds on prior research comparing three prompting strategies for natural-language command parsing into JSON schemas. Expanding that framework, the current work investigates how adapter-based fine-tuning, where most model parameters are frozen and only small adapter modules are trained, affects model accuracy, consistency, and robustness. The experiment uses the same controlled shopping-cart parsing task and dataset of 1000 synthetic commands to ensure direct comparability. Results will quantify the trade-off between computational cost and performance gains, offering evidence-based insights into whether fine-tuning is a justified investment compared to advanced prompt engineering. The expected contribution is a clear, empirical framework for deciding when fine-tuning meaningfully enhances LLM utility in applied natural-language understanding.

Keywords: LLM, Adapters Fine-Tuning, Prompt Engineering, Natural Language Understanding, JSON Parsing, Structured Output, Model Performance

1. Introduction

The rapid evolution of Large Language Models (LLMs) has fundamentally transformed the landscape of Natural Language Processing (NLP), enabling advanced capabilities in reasoning, content generation, and semantic understanding [1]. While early applications focused primarily on open-ended text generation, modern production environments increasingly require LLMs to function as reliable semantic parsers capable of mapping unstructured user intent into rigorous, machine-readable formats such as JSON or SQL [2]. This capability is particularly critical in domain-specific applications, such as e-commerce controllers, where valid schema adherence is a prerequisite for system functionality. A primary challenge in deploying these systems is the "optimization dilemma": choosing between inference-time strategies and model training. As highlighted in our previous survey [3], while prompt engineering offers a lightweight "black-box" approach to adaptation, it often struggles with robustness when constrained to rigid schema definitions. Conversely, full-parameter fine-tuning yields high precision but incurs prohibitive computational costs and memory requirements [4]. To address this, Parameter-Efficient

Received:
Revised:
Accepted:
Published:

Fine-Tuning (PEFT) techniques, such as Low-Rank Adaptation (LoRA), have emerged as a middle ground, freezing pre-trained weights and injecting trainable rank-decomposition matrices to approximate full fine-tuning performance with a fraction of the resources [5]. To address these challenges, this study evaluates the cost-benefit ratio of fine-tuning against prompting for high-fidelity semantic parsing. Building upon the theoretical frameworks established in our prior work [3], we implement a controlled experimental design using the LLaMA-3-8B model. The system compares two distinct optimization paradigms: Advanced Prompt Engineering and Adapter-Based Fine-Tuning, utilizing LoRA and its quantized variant (QLoRA) to update the model's internal representations [6]. This setup allows for a direct comparison of how each method handles the rigorous constraints of mapping natural language commands to structured JSON outputs. The main aim of this research is to identify the "break-even point" where the computational investment of training adapters yields a statistically significant advantage over advanced prompting. The main contribution of this study is a structured empirical framework for benchmarking LLaMA-3 on structured parsing tasks, providing evidence-based insights into whether the resource consumption of fine-tuning is justified by gains in accuracy and robustness. The evaluation reveals that while prompting is sufficient for simple intents, adapter-based methods significantly outperform in handling complex, multi-intent schemas, particularly when targeting all linear layers with optimized hyperparameters.

2. Background and Related Works

The rapid growth of large language models (LLMs) has necessitated strategies for adapting them efficiently to downstream tasks. Zhiqiang Hu et al. [5] introduced LLM-Adapters, a framework for parameter-efficient fine-tuning (PEFT) that integrates Series adapters, Parallel adapters, Prefix-Tuning, and LoRA. Their study demonstrated that smaller LLaMA models (7B and 13B) equipped with optimized adapters could match or surpass the performance of much larger models such as ChatGPT and GPT-3.5, particularly on structured reasoning tasks using datasets like Math10K and Commonsense170K. These findings suggest that adapter-based PEFT can improve structured task accuracy compared to advanced prompting strategies.

Ding et al. [7] proposed a unified framework called delta-tuning, which adjusts only a small fraction of model parameters. Through extensive evaluation on over 100 NLP tasks, they found that delta-tuning achieved performance comparable to full fine-tuning while significantly reducing GPU memory usage and accelerating backpropagation. Larger models not only benefited more from minimal parameter updates but also converged faster, reinforcing the potential of adapter-based methods to enhance task-specific performance efficiently.

Esmaeili et al. [8] conducted empirical studies on PEFT for code-specific LLMs, examining LoRA, Compacter, and IA3. Their findings showed that LoRA consistently outperformed other adapters in code summarization and generation, occasionally exceeding full fine-tuning on low-resource languages like R. The study highlighted that the number of trainable parameters influenced functional correctness more than the adapter architecture itself, further supporting the effectiveness of PEFT in improving structured outputs in specialized domains.

Razuvayevskaya et al. [9] compared parameter-efficient techniques and full fine-tuning for multilingual news classification. Using XLM-RoBERTa Large, they showed that LoRA and bottleneck adapters could reduce trainable parameters by 140–280 times while maintaining competitive accuracy. Their results suggested that PEFT strategies can outperform traditional prompting approaches, especially when sufficient source data is available.

Shin et al. [10] assessed the trade-offs between prompt engineering and fine-tuning for automated code tasks. Their evaluation of GPT-4 against seventeen fine-tuned baselines revealed that while task-specific prompting sometimes outperformed automated methods in code summarization, fine-tuned models were generally superior in code generation. Human-in-the-loop conversational prompting further enhanced outcomes, demonstrating that adapter-based fine-tuning can stabilize and improve output quality over prompting alone.

Ming Gong et al. [11] developed structure-learnable adapters, which introduced differentiable gating and sparsity control to dynamically optimize adapter placement and activation paths. Experiments on the Multi-Task NLU Benchmark indicated that this approach not only achieved high accuracy with minimal parameters but also reduced output variability and maintained robustness under noisy conditions, supporting the notion that adapter fine-tuning can yield more consistent outputs than prompting.

Fang and Ye [12] proposed RFedLR, a robust PEFT framework for federated learning. By selectively updating noise-sensitive parameters and dynamically weighting client updates, RFedLR achieved up to 10.15% accuracy improvements over state-of-the-art baselines under high-noise conditions while using only 0.1754% of the trainable parameters. These results reinforce that adapter-based fine-tuning provides more stable and reliable outputs than prompt-based methods, even in decentralized and noisy environments.

Shuo Chen et al. [13] benchmarked eleven adaptation methods on vision-language models under textual and visual corruptions. They found that parameter-efficient adapters exhibited higher resilience to input perturbations than full fine-tuning or prompting, although no single method dominated across all datasets. Increasing adaptation data or parameter size did not guarantee robustness, highlighting the importance of careful PEFT design to enhance model stability.

Chen et al. [14] evaluated prompt engineering for industrial document processing tasks and found that few-shot learning improved reasoning capabilities over zero-shot prompts. However, adapter-based PEFT consistently offered more reliable and less variable outputs, particularly in complex scenarios with noisy OCR inputs.

Kaijie Zhu et al. [15] introduced PromptRobust, a benchmark for adversarial prompt evaluation across multiple LLMs. Their results showed that word-level adversarial prompts caused a 39% average performance drop, and few-shot prompts exhibited better robustness than zero-shot prompting. The study illustrates that adapter fine-tuning can improve resilience to prompt-based perturbations.

Jaehyung Kim et al. [16] presented ROAST, combining adversarial perturbations with selective training to improve robustness across sentiment classification and entailment tasks. ROAST yielded average improvements of 18.39% and 7.63% over state-of-the-art baselines, demonstrating that adapter fine-tuning can mitigate input noise effects more effectively than prompting.

Pei et al. [17] proposed SelfPrompt, which autonomously evaluates LLM robustness using domain-constrained knowledge graphs and adversarial prompt generation. Their findings indicated that domain-specific adapter fine-tuning enhances robustness in specialized fields like medicine and biology, outperforming prompting strategies alone.

Lin Mu et al. [18] introduced Robustness of Prompting (RoP), a parameter-free method for improving LLM resilience through error correction and guidance prompts. While effective, experiments confirmed that adapter fine-tuning provides superior stability and transferability across architectures, emphasizing the practical value of PEFT in noisy real-world environments.

Sajjadi Mohammadabadi et al. [19] surveyed LLM architectures and adaptation methods, emphasizing the efficiency of PEFT, instruction tuning, and RLHF. They noted that

adapter-based fine-tuning can deliver significant performance gains relative to prompting, justifying its computational cost in applied settings.

Trad and Chehab [20] studied phishing detection LLMs and found that while refined prompting improved performance, fine-tuned models achieved higher F1-scores and superior reliability, demonstrating that the cost of PEFT is justified when high precision is required.

Pornprasit and Tantithamthavorn [21] evaluated LLM-based code review, showing that fine-tuning GPT-3.5 achieved 73–74% higher Exact Match rates than prompting approaches. Their study confirmed that adapter-based fine-tuning can deliver meaningful gains even with modest training data.

Wang et al. [22] introduced COSMOS, a framework for predicting adaptation outcomes with minimal trials. Their results indicated that fine-tuning consistently outperformed in-context learning in medium to high-cost scenarios, supporting the view that adapter PEFT justifies its resource investment by providing predictable and superior performance.

Based on the cumulative evidence from these studies, the overarching goal of this work is to investigate how adapter-based parameter-efficient fine-tuning (PEFT) compares to advanced prompting in terms of structured task accuracy, output stability, robustness, and cost-effectiveness. Accordingly, the following research hypotheses are formulated:

- H1: Adapter-based PEFT improves structured task performance relative to advanced prompting.
- H2: Adapter fine-tuning reduces output variability compared to prompting approaches.
- H3: Adapter fine-tuning enhances robustness to input noise, adversarial perturbations, and paraphrasing relative to prompting.
- H4: Performance gains achieved via adapter-based PEFT justify the computational and resource cost compared to advanced prompting methods.

3. Methodology

This section describes the experimental framework designed to evaluate the trade-offs between parameter-efficient fine-tuning (PEFT) and advanced in-context learning (ICL) for structured information extraction. The primary objective is to measure performance, structural reliability, and generalization capability of large language models (LLMs) when converting unstructured natural language instructions into strictly formatted JSON objects. The methodology extends the comparative benchmarking approach of [3] to a controlled structured parsing domain.

3.1. Task Definition

The experimental task focuses on Natural Language to Structured Command conversion, a core component of automated commerce and inventory systems. Given a free-form shopping instruction, the model must output a valid JSON object with the following schema:

```
{
  "action": "...",
  "quantity": "...",
  "product": "..."
}
```

Each instance requires extraction or inference of three attributes:

The experimental task focuses on the "Natural Language to Structured Command" conversion, a critical component in automated inventory and e-commerce systems. To

rigorously evaluate the effectiveness of both prompting and adapter-based fine-tuning, we constructed a controlled dataset comprising 10,000 annotated natural-language shopping instructions. To ensure high-quality and diverse data, samples were synthetically generated using a teacher-model paradigm.

Each instance requires the model to extract or infer three specific attributes and map them deterministically to a fixed JSON schema:

1. Action: Binary classification over {add, remove}.
2. Product: Named entity extraction identifying the canonical product name.
3. Quantity: Integer extraction with a default value of 1 when no explicit quantity is provided.

The mapping from natural language to JSON is deterministic, enabling objective evaluation of both semantic correctness and structural integrity.

3.2. Dataset Construction

The dataset consists of 12,000 synthetically generated but linguistically diverse shopping instructions, created in collaboration with industry partners to reflect realistic user interaction patterns.

Although synthetically produced, the corpus incorporates substantial variation to prevent surface-form memorization. Diversity mechanisms include:

3.2.1. Dataset Splitting for Generalization Assessment

To balance computational feasibility with sufficient diversity for adapter specialization, the 10,000 examples were partitioned into three disjoint subsets:

- Verb paraphrasing (e.g., add, insert, remove, delete, take out)
- Explicit and implicit quantity expressions
- Variable grammatical constructions
- Multi-word paraphrases and lexical substitutions

Each example is paired with a normalized ground-truth JSON annotation containing the three labeled attributes.

3.3. Data Splits

The corpus is partitioned as follows:

Based on empirical evidence identifying optimal configurations for LLaMA-based architectures, we targeted all linear modules within the transformer architecture (specifically W_q, W_k, W_v, W_o, and MLP layers) to ensure the adapter captured the necessary structural constraints for JSON generation. The specific hyperparameters utilized include:

- Training Set: 8,000 examples used for adapter parameter optimization.
- Validation Set: 2,000 examples used for monitoring convergence and preventing overfitting.
- Test Set: 2,000 disjoint examples reserved exclusively for final evaluation.

The test set remains completely unseen during adapter training and validation. It contains novel paraphrasing patterns and lexical combinations to assess generalization beyond the training distribution.

3.4. Model Selection

To analyze the effect of model scale and architectural variation, we evaluate a heterogeneous suite of LLMs across parameter sizes and training paradigms. We have selected three models that represent a range of scales and architectural designs, all of which are compatible with adapter-based fine-tuning:

Table 1. Evaluated models and their parameter counts

Name	Size
Llama 3.1	8B
Qwen 3	4B
Llama 3.2	3B

All models are evaluated under identical inference conditions with temperature set to 0 to eliminate stochastic variance.

3.5. Parameter-Efficient Fine-Tuning

For each base model, we compare three configurations:

1. Base Model (Prompting Only): No parameter updates; task performed via structured prompt engineering.
2. LoRA Adapter: Low-Rank Adaptation modules trained on the 8,000-example training set.
3. IA^3 Adapter: Infused Adapter by Inhibiting and Amplifying Inner Activations trained under identical conditions.

Adapters are implemented using the PEFT framework within the Hugging Face Transformers ecosystem. Hyperparameters, tokenizer configuration, and preprocessing steps are held constant across experiments to ensure fair comparison. No hyperparameter search is performed.

Training is conducted using multi-GPU Distributed Data Parallel (DDP).

QLoRA is not employed in this study.

3.6. Evaluation Protocol

All configurations are evaluated on the same held-out test set of 2,000 examples.

A prediction is considered correct only if:

- All three fields (action, quantity, product) exactly match the ground truth.
- The output is valid JSON and can be successfully parsed.

If the model fails to produce valid JSON, the prediction is counted as incorrect.

Performance is measured using Precision, Recall, and F1 Score under strict exact-match criteria. F1 Score serves as the primary comparison metric.

3.7. Comparative Analysis

For each model, we report results in the following format:

Model	Base (Prompt)	LoRA	IA^3
...	...	...	...

This structure enables direct comparison between prompt-based inference and adapter-based specialization across model scales.

3.8. Adapter-Based Fine-Tuning (LoRA)

For the fine-tuning paradigm, we employed Low-Rank Adaptation (LoRA). This technique was chosen for its ability to maintain the underlying general knowledge of the base model while specializing its output structure for JSON parsing with minimal computational overhead.

The LoRA approach modifies the pre-trained weight matrices $W_0 \in \mathbb{R}^{d \times k}$ by adding a low-rank decomposition BA, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$. During the forward pass,

the output of the adapter is scaled by a factor of $\frac{\alpha}{r}$ to balance the retention of pre-trained information with the targeted domain adaptation. The forward pass for a given input x is thus represented as:

$$h = W_0 x + \left(\frac{\alpha}{r}\right) \Delta W x = W_0 x + \left(\frac{\alpha}{r}\right) BAx \quad (1)$$

In this study, we targeted all linear modules within the transformer architecture to ensure the adapter captured the necessary structural and semantic constraints for JSON generation. Specifically, the adaptation was applied comprehensively to both the attention mechanisms (`q_proj`, `k_proj`, `v_proj`, `o_proj`) and the feed-forward network layers (`gate_proj`, `up_proj`, `down_proj`). The specific LoRA configuration utilized during fine-tuning included:

- Rank (r): 32. A higher rank was selected to provide sufficient parameter capacity for the complex semantic mapping required in structured parsing tasks.
- Alpha (α): 64. The scaling factor was strictly set to $\alpha = 2r$, a heuristic demonstrated to stabilize training and improve convergence when utilizing higher ranks.
- Dropout: 0.05, applied to the adapter layers as a regularization measure to prevent overfitting.
- Precision and Compute: Models were trained without 4-bit quantization, instead utilizing BFloat16 (`bf16`) mixed precision and TensorFloat-32 (`tf32`) optimizations natively supported by NVIDIA A100 GPUs to accelerate matrix multiplications.

3.8.1. Optimization and Training Dynamics

The Supervised Fine-Tuning (SFT) phase was executed over 3 epochs with a maximum sequence length of 1024 tokens. To optimize memory footprint during backpropagation, gradient checkpointing was enabled. The training pipeline utilized a fused AdamW optimizer (`adamw_torch_fused`) with a peak learning rate of 2×10^{-4}, a weight decay of 0.01, and a warm-up ratio of 3% to ensure early-stage stability. An effective batch size of 32 was maintained by utilizing a per-device batch size of 8 combined with 4 gradient accumulation steps.

3.9. Prompt Engineering Strategies

To establish a baseline for In-Context Learning (ICL), we implemented three escalating levels of prompt complexity:

1. Minimal Instruction (Zero-Shot): A concise prompt defining the role of the model and the required JSON schema.
2. Extended Prompt: This strategy includes detailed edge-case instructions, emphasizing the default quantity logic and strict "no-prose" constraints to prevent conversational "chatter."
3. Few-Shot Prompting: Building upon the Extended Prompt, this version includes [Insert Number] high-quality examples of natural language inputs and their corresponding JSON outputs, providing a semantic and structural blueprint for the model.

3.10. Evaluation Metrics

The models were evaluated on two primary dimensions to assess both syntactic integrity and semantic accuracy.

3.10.1. JSON Validity Rate

Before measuring accuracy, we assess if the output is a "well-formed" JSON object. Any output that includes conversational filler, markdown errors, or fails standard library

parsing (e.g., Python's `json.loads()`) is categorized as a failure. This metric directly evaluates the model's susceptibility to structural hallucination.

3.10.2. Structured F1 Score

For all valid JSON outputs, we calculate the F1 score. This requires an exact match between the predicted value and the ground truth for each of the three fields (Action, Product, Quantity). The field-level F1 score is defined as the harmonic mean of precision (P) and recall (R):

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \tag{2}$$

We report the macro-averaged F1 across all fields to provide a singular, robust measure of parsing performance.

4. Results and discussion

5. Conclusion

6. Acknowledgments

This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0028 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136, and UNIN-TEH-25-1-8.

7. Results

This section may be divided by subheadings. It should provide a concise and precise description of the experimental results, their interpretation as well as the experimental conclusions that can be drawn.

7.1. Subsection

7.1.1. Subsubsection

Bulleted lists look like this:

- First bullet;
- Second bullet;
- Third bullet.

Numbered lists can be added as follows:

1. First item;
2. Second item;
3. Third item.

The text continues here.

7.2. Figures, Tables and Schemes

All figures and tables should be cited in the main text as Figure 1, Table 2, etc.

Figure 1. This is a figure. Schemes follow the same formatting.

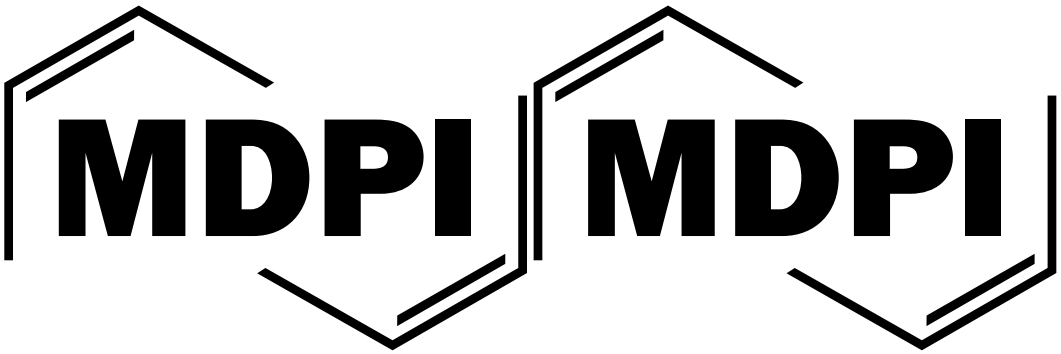
Table 2. This is a table caption. Tables should be placed in the main text near to the first time they are cited.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data [1]

[1] Tables may have a footer.

The text continues here (Figure 2 and Table 3).

(**a**) (**b**)



(**c**) (**d**)

Figure 2. This is a wide figure. Schemes follow the same formatting. If there are multiple panels, they should be listed as: (**a**) Description of what is contained in the first panel. (**b**) Description of what is contained in the second panel. (**c**) Description of what is contained in the third panel. (**d**) Description of what is contained in the fourth panel. Figures should be placed in the main text near to the first time they are cited. A caption on a single line should be centered.

Table 3. This is a wide table.

Title 1	Title 2	Title 3	Title 4
Entry 1 *	Data Data Data	Data Data Data	Data Data Data
Entry 2	Data Data Data	Data Data Data	Data Data Data

* Tables may have a footer.

Text.

Text.

7.3. Formatting of Mathematical Components

This is the example 1 of equation:

$$a = 1, \tag{3}$$

the text following an equation need not be a new paragraph. Please punctuate equations as regular text.

This is the example 2 of equation:

$$a = b + c + d + e + f + g + h + i + j + k + l + m + n + o + p + q + r + s + t + u + v + w + x + y + z \tag{4}$$

Please punctuate equations as regular text. Theorem-type environments (including propositions, lemmas, corollaries etc.) can be formatted as follows:

Theorem 1. *Example text of a theorem.*

The text continues here. Proofs must be formatted as follows:

Proof of Theorem 1. Text of the proof. Note that the phrase "of Theorem 1" is optional if it is clear which theorem is being referred to. □

The text continues here.

8. Discussion

Authors should discuss the results and how they can be interpreted from the perspective of previous studies and of the working hypotheses. The findings and their implications should be discussed in the broadest context possible. Future research directions may also be highlighted.

9. Conclusions

This section is not mandatory, but can be added to the manuscript if the discussion is unusually long or complex.

10. Patents

This section is not mandatory, but may be added if there are patents resulting from the work reported in this manuscript.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used "Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.", please turn to the CRediT taxonomy for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: Please add: "This research received no external funding" or "This research was funded by NAME OF FUNDER grant number XXX." and and "The APC was funded by XXX". Check carefully that the details given are accurate and use the standard spelling of funding agency names at https://search.crossref.org/funding, any errors may affect your future funding.

Institutional Review Board Statement: In this section, you should add the Institutional Review Board Statement and approval number, if relevant to your study. You might choose to exclude this statement if the study did not require ethical approval. Please note that the Editorial Office might ask

you for further information. Please add "The study was conducted in accordance with the Declaration of Helsinki, and approved by the Institutional Review Board (or Ethics Committee) of NAME OF INSTITUTE (protocol code XXX and date of approval)." for studies involving humans. OR "The animal study protocol was approved by the Institutional Review Board (or Ethics Committee) of NAME OF INSTITUTE (protocol code XXX and date of approval)." for studies involving animals. OR "Ethical review and approval were waived for this study due to REASON (please provide a detailed justification)." OR "Not applicable" for studies not involving humans or animals.

Informed Consent Statement: Any research article describing a study involving humans should contain this statement. Please add "Informed consent was obtained from all subjects involved in the study." OR "Patient consent was waived due to REASON (please provide a detailed justification)." OR "Not applicable" for studies not involving humans. You might also choose to exclude this statement if the study did not involve humans.

Written informed consent for publication must be obtained from participating patients who can be identified (including by the patients themselves). Please state "Written informed consent has been obtained from the patient(s) to publish this paper" if applicable.

Data Availability Statement: We encourage all authors of articles published in MDPI journals to share their research data. In this section, please provide details regarding where data supporting reported results can be found, including links to publicly archived datasets analyzed or generated during the study. Where no new data were created, or where data is unavailable due to privacy or ethical restrictions, a statement is still required. Suggested Data Availability Statements are available in section "MDPI Research Data Policies" at https://www.mdpi.com/ethics.

Acknowledgments: In this section you can acknowledge any support given which is not covered by the author contribution or funding sections. This may include administrative and technical support, or donations in kind (e.g., materials used for experiments). Where GenAI has been used for purposes such as generating text, data, or graphics, or for study design, data collection, analysis, or interpretation of data, please add "During the preparation of this manuscript/study, the author(s) used [tool name, version information] for the purposes of [description of use]. The authors have reviewed and edited the output and take full responsibility for the content of this publication."

Conflicts of Interest: Declare conflicts of interest or state "The authors declare no conflicts of interest." Authors must identify and declare any personal circumstances or interest that may be perceived as inappropriately influencing the representation or interpretation of reported research results. Any role of the funders in the design of the study; in the collection, analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results must be declared in this section. If there is no role, please state "The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results".

Abbreviations

The following abbreviations are used in this manuscript:

MDPI	Multidisciplinary Digital Publishing Institute
DOAJ	Directory of open access journals
TLA	Three letter acronym
LD	Linear dichroism

Appendix A

Appendix A.1

The appendix is an optional section that can contain details and data supplemental to the main text—for example, explanations of experimental details that would disrupt the flow of the main text but nonetheless remain crucial to understanding and reproducing the research shown; figures of replicates for experiments of which representative data are

shown in the main text can be added here if brief, or as Supplementary Data. Mathematical proofs of results not central to the paper can be added as an appendix.

Table A1. This is a table caption.

Title 1	Title 2	Title 3
Entry 1	Data	Data
Entry 2	Data	Data

Appendix B

All appendix sections must be cited in the main text. In the appendices, Figures, Tables, etc. should be labeled, starting with "A"—e.g., Figure A1, Figure A2, etc.

References

1. Ling, C.; Zhao, X.; Lu, J.; Deng, C.; Zheng, C.; Wang, J.; Chowdhury, T.; Li, Y.; Cui, H.; Zhang, X.; et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys* **2025**, *58*, 1–39.
2. Han, Z.; Gao, C.; Liu, J.; Zhang, J.; Zhang, S.Q. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608* **2024**.
3. Karlović, R.; Lorencin, I. Large language models as Retail Cart Assistants: A Prompt-Based Evaluation. *Human Systems Engineering and Design (IHSED2025): Future Trends and Applications* **2025**, *198*.
4. Mundra, N.; Doddapaneni, S.; Dabre, R.; Kunchukuttan, A.; Puduppully, R.; Khapra, M.M. A comprehensive analysis of adapter efficiency. In Proceedings of the Proceedings of the 7th Joint International Conference on Data Science & Management of Data (11th ACM IKDD CODS and 29th COMAD), 2024, pp. 136–154.
5. Hu, Z.; Wang, L.; Lan, Y.; Xu, W.; Lim, E.P.; Bing, L.; Xu, X.; Poria, S.; Lee, R. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In Proceedings of the Proceedings of the 2023 conference on empirical methods in natural language processing, 2023, pp. 5254–5276.
6. Patil, R.; Khot, P.; Gudivada, V. Analyzing LLAMA3 Performance on Classification Task Using LoRA and QLoRA Techniques. *Applied Sciences* **2025**, *15*, 3087.
7. Ding, N.; Qin, Y.; Yang, G.; Wei, F.; Yang, Z.; Su, Y.; Hu, S.; Chen, Y.; Chan, C.M.; Chen, W.; et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence* **2023**, *5*, 220–235.
8. Esmaeili, A.; Saberi, I.; Fard, F. Empirical studies of parameter efficient methods for large language models of code and knowledge transfer to r. *Empirical Software Engineering* **2026**, *31*, 30.
9. Razuvayevskaya, O.; Wu, B.; Leite, J.A.; Heppell, F.; Srba, I.; Scarton, C.; Bontcheva, K.; Song, X. Comparison between parameter-efficient techniques and full fine-tuning: A case study on multilingual news article classification. *Plos one* **2024**, *19*, e0301738.
10. Shin, J.; Tang, C.; Mohati, T.; Nayebi, M.; Wang, S.; Hemmati, H. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. In Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR). IEEE, 2025, pp. 490–502.
11. Gong, M.; Deng, Y.; Qi, N.; Zou, Y.; Xue, Z.; Zi, Y. Structure-learnable adapter fine-tuning for parameter-efficient large language models. In Proceedings of the International Conference on Artificial Intelligence and Computational Engineering (AICE 2025). IET, 2025, Vol. 2025, pp. 225–230.
12. Fang, X.; Ye, M. Towards Robust Parameter-Efficient Fine-Tuning for Federated Learning. In Proceedings of the The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025.
13. Chen, S.; Gu, J.; Han, Z.; Ma, Y.; Torr, P.; Tresp, V. Benchmarking robustness of adaptation methods on pre-trained vision-language models. *Advances in Neural Information Processing Systems* **2023**, *36*, 51758–51777.
14. Chen, L.C.; Weng, H.T.; Pardeshi, M.S.; Chen, C.M.; Sheu, R.K.; Pai, K.C. Evaluation of Prompt Engineering on the Performance of a Large Language Model in Document Information Extraction. *Electronics* **2025**, *14*, 2145.
15. Zhu, K.; Wang, J.; Zhou, J.; Wang, Z.; Chen, H.; Wang, Y.; Yang, L.; Ye, W.; Zhang, Y.; Gong, N.; et al. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In Proceedings of the Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis, 2023, pp. 57–68.
16. Kim, J.; Mao, Y.; Hou, R.; Yu, H.; Liang, D.; Fung, P.; Wang, Q.; Feng, F.; Huang, L.; Khabsa, M. RoAST: Robustifying Language Models via Adversarial Perturbation with Selective Training. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 3412–3444.

17. Pei, A.; Yang, Z.; Zhu, S.; Cheng, R.; Jia, J. Selfprompt: Autonomously evaluating llm robustness via domain-constrained knowledge guidelines and refined adversarial prompts. In Proceedings of the Proceedings of the 31st International Conference on Computational Linguistics, 2025, pp. 6840–6854.
18. Mu, L.; Chu, G.; Ni, L.; Sang, L.; Wu, Z.; Jin, P.; Zhang, Y. Robustness of Prompting: Enhancing Robustness of Large Language Models Against Prompting Attacks. *arXiv preprint arXiv:2506.03627* **2025**.
19. Sajjadi Mohammadabadi, S.M.; Kara, B.C.; Eyupoglu, C.; Uzay, C.; Tosun, M.S.; Karakuş, O. A survey of large language models: evolution, architectures, adaptation, benchmarking, applications, challenges, and societal implications. *Electronics* **2025**, *14*.
20. Trad, F.; Chehab, A. Prompt engineering or fine-tuning? a case study on phishing detection with large language models. *Machine Learning and Knowledge Extraction* **2024**, *6*, 367–384.
21. Pornprasit, C.; Tantithamthavorn, C. Fine-tuning and prompt engineering for large language models-based code review automation. *Information and Software Technology* **2024**, *175*, 107523.
22. Wang, J.; Albarghouthi, A.; Sala, F. COSMOS: Predictable and Cost-Effective Adaptation of LLMs. *arXiv preprint arXiv:2505.01449* **2025**.